

Rozwój aplikacji do analizy danych uzyskanych w eksperymencie Warsaw TPC. – raport

Przemysław Szyc
Jan Kierzkowski
Filip Baran
Mateusz Perc
Krzysztof Macuk

Opiekun: dr hab. Artur Kalinowski

10 lutego 2025

1 Wstęp

Analiza danych w eksperymencie WarsawTPC [1], zajmującym się badaniem zjawiska fotodezintegracji atomów tlenu i węgla odbywa się przy pomocy programów z repozytorium TPCReco [2], napisanych w języku C++. Istotnym elementem tego oprogramowania są symulacje zdarzeń metodą Monte Carlo, planowane jest też rozbudowanie aplikacji o moduł uczenia maszynowego. Rozwój aplikacji TPCReco pozwoli na sprawniejszą analizę danych oraz dokładniejsze zbadanie wspomnianego zjawiska.

2 Cel projektu zespołowego

Projekt zespołowy zakładał rozszerzenie funkcjonalności oprogramowania TPCReco. Jednym z zakładanych celów była integracja modułu MonteCarlo z interfejsem graficznym aplikacji poprzez stworzenie nowej klasy generującej przypadki.

Ponadto, planowano opracowanie metody uruchamiania modeli Tensorflow[3] w środowisku aplikacji, a następnie rozwój modułu rekonstrukcji przypadków przy pomocy sieci neuronowych.

Dodatkowym celem projektu było stworzenie dedykowanego pliku Dockerfile, który zapewni niezależność od prywatnego obrazu **elitpc/get** i umożliwi łatwiejszą dystrybucję oraz wdrażanie oprogramowania. Aby zapewnić wysoką jakość kodu, zaplanowano wdrożenie mechanizmów automatycznego testowania kodu z wykorzystaniem GitHub Actions[4] obejmujących uruchamianie testów jednostkowych oraz analizę poprawności kodu.

Dodatkowym zadaniem było utworzenie bazy danych do przechowywania i analizy parametrów pomiarowych związanych z przebiegami eksperymentu.

Ostatnim zadaniem zespołu było rozwijanie oraz dodawanie testów jednostkowych do aplikacji.

3 Repozytoria i obszary robocze projektu

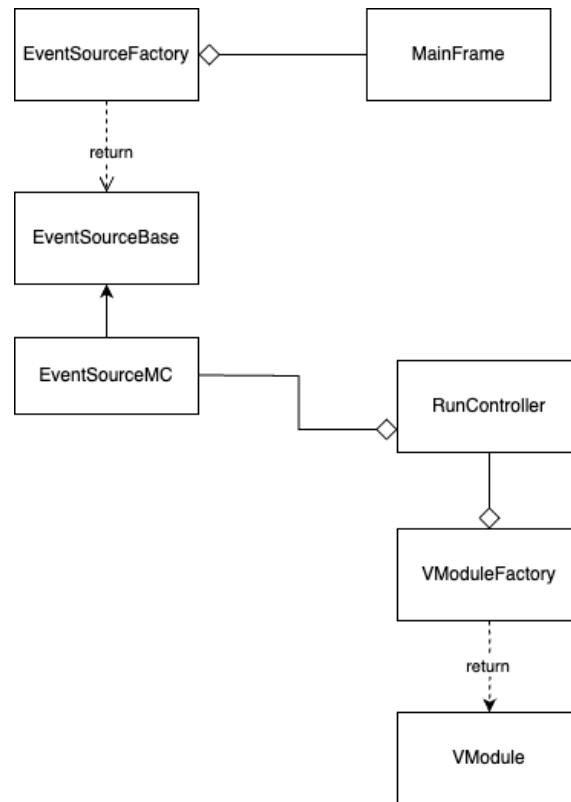
Kod aplikacji jest dostępny w repozytorium w serwisie Github [2]. Praca zespołu odbywała się w repozytrium będącym forkiem projektu [5].

Ponadto zespół korzystał z przestrzeni w Notion [6] na potrzeby związane z tworzeniem i podziałem zadań, przekazywaniem raportów częściowych opiekunowi projektu, oraz spisywaniem instrukcji do niektórych czynności.

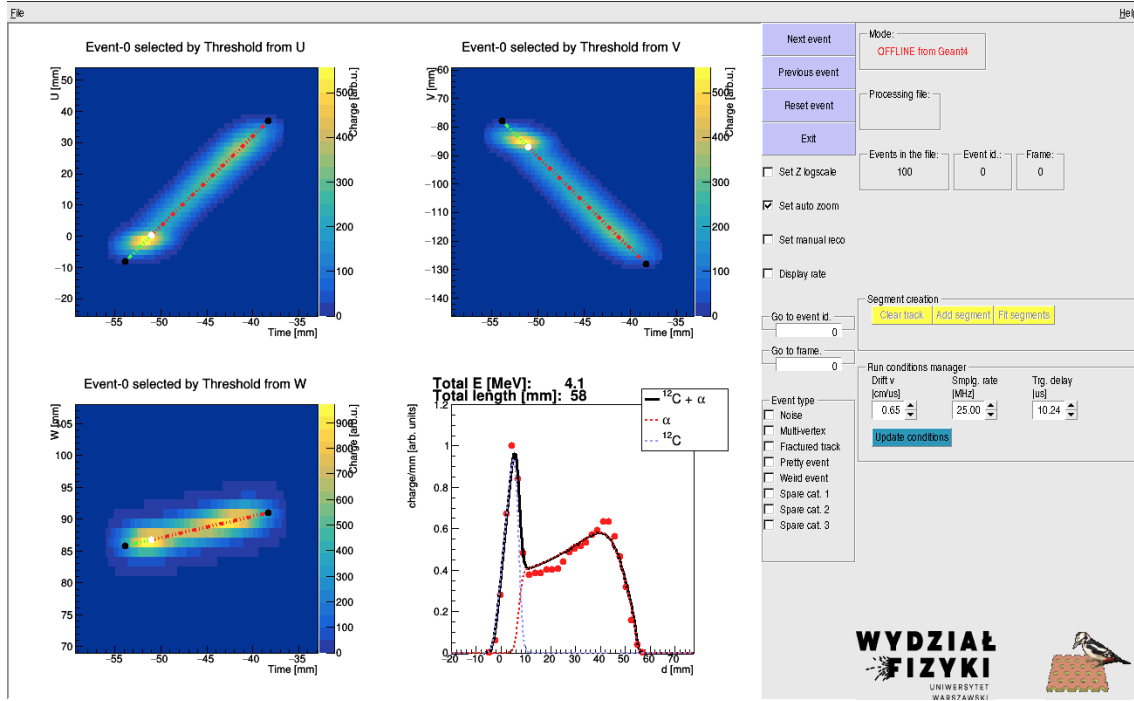
4 Zmiany klasy EventSourceMC

W ramach projektu wykonane zostało połączenie pomiędzy modułem MonteCarlo oraz klasą MainFrame z modułu GUI. Było ono możliwe dzięki wprowadzeniu zmian w klasie EventSourceMC. Logika tworzenia nowych przypadków przed zmianami była bezpośrednio zaimplementowana w

metodach tej klasy. Nowe rozwiązanie polega na oddelegowaniu procesu tworzenia nowych przypadków klasie RunController, który służy do uruchamiania sekwencyjnie sub-modułów MonteCarlo. Wprowadzone zmiany są zgodne z zasadami programowania obiektowego, w szczególności z zasadą pojedynczej odpowiedzialności [7] oraz umożliwiają lepszą organizację kodu. Konsekwencji wykonanego połączenia jest też nowa funkcjonalność w postaci tego, że tworzone i oglądane przypadki mogą być zapisywane do pliku jeśli w sekwencji modułów pojawi się sub-moduł EventFileExporter.



Rysunek 1: Diagram UML zaimplementowanego rozwiązania. Wykonane zostało połączenie między EventSourceMC oraz RunController-em. Celem tego połączenia było wykorzystanie logiki symulacji przypadków zaimplementowanej w module MonteCarlo.

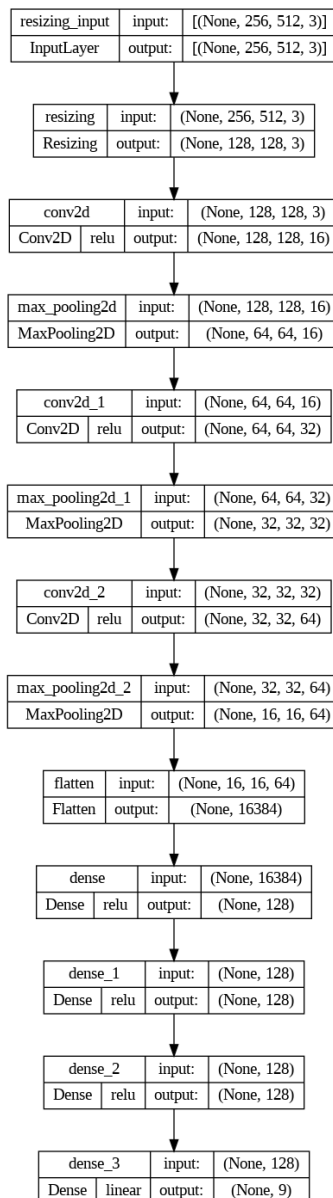


Rysunek 2: Widok GUI w przypadku wykorzystanie źródła przypadków EventSourceMC. Nowa konfiguracja wyświetla informacje o trybie działania: "OFFLINE from Geant4".

5 Uruchamianie modeli Tensorflow

5.1 Rezultaty

W ramach projektu planowano stworzyć moduł rekonstrukcji zdarzeń przy pomocy uczenia maszynowego. Podjęto kilka prób znalezienia najlepszego sposobu uruchamiania modeli Tensorflow w języku w którym napisana jest aplikacja - C++. Sprawdzono bibliotekę [8] która będąc popularnym rozwiązaniem nie była jednak możliwa do uruchomienia w kontenerze developerskim. Ostatecznie, zdecydowano się posłużyć API C do Tensorflow. Jest ono prostsze w obsłudze zarówno na poziomie kodu aplikacji jak i samego Dockerfile. API C++ również mogłoby zostać użyte, jednak poszukiwania zostały przerwane gdy zespół znalazł działający przykład oraz dostosował go do najnowszej wersji Tensorflow.



Rysunek 3: Schemat modelu uruchomionego poprzez API C do Tensorflow.

5.2 Sugerowane kroki

Sugerowanym kolejnym krokiem jest integracja stworzonego rozwiązania z resztą aplikacji poprzez stworzenie klasy MLTrackBuilder.

6 Dockerfile aplikacji

6.1 Rezultaty

Stworzony został plik Dockerfile dla aplikacji w oparciu o Ubuntu 16.04 z wersją programu Root 6.08. Plik ten posłużyć może do tworzenia nowego obrazu podstawowego który zamienić może prywatny obraz elitpc/get używany do tej pory. Wykorzystując nowy obraz bazowy zbudowano aplikację oraz uruchomiono z powodzeniem testu. Nowe rozwiązanie nie pozwala jednak na uruchamianie aplikacji z plikami GRAW.

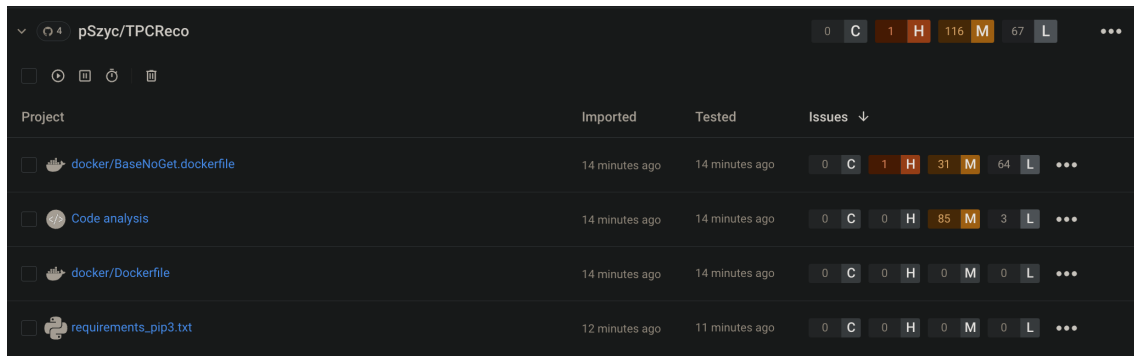
6.2 Sugerowane kroki

Należałoby stworzyć kontener w oparciu o najnowszy obraz Ubuntu oraz nowsze wersje programów Geant4 oraz Root. Posiadanie publicznego Dockerfile umożliwi ponadto budowanie aplikacji w Github Actions.

7 Github actions

7.1 Rezultaty

Zaimplementowano dwie funkcjonalności przy użyciu Github Actions [4]: automatyczne uruchamianie testów jednostkowych oraz skan bezpieczeństwa przy pomocy GitLeaks[9] i Snyk [10]. Uruchamianie testów jednostkowych w obliczu braku Dockerfile lub innej metody umożliwiającej zbudowanie aplikacji zostało zaimplementowane przez pobieranie kontenera z Google Drive. W obecnej konfiguracji przy wykorzystaniu singularity w Github Actions z powodzeniem uruchamiane są testy jednostkowe. Zastosowany skan GitLeaks nie wykrył niebezpieczeństw. Narzędzie Snyk skanując kod poleciło zwrócenie uwagi na kwestie sprawdzania zawartości wczytywanych przez aplikację plików, zwalniania alokowanej pamięci oraz poprawienie pliku Dockerfile.



The screenshot shows the Snyk interface for a repository named 'pSzyc/TPCReco'. At the top, there is a summary bar with counts: 0 Critical (C), 1 High (H), 116 Medium (M), and 67 Low (L). Below this, a table lists the scanned files. Each row includes a file icon, the file name, the time it was imported and tested, and a detailed breakdown of issue counts by severity.

Project	Imported	Tested	Issues
docker/BaseNoGet.dockerfile	14 minutes ago	14 minutes ago	0 C, 1 H, 31 M, 64 L
Code analysis	14 minutes ago	14 minutes ago	0 C, 0 H, 85 M, 3 L
docker/Dockerfile	14 minutes ago	14 minutes ago	0 C, 0 H, 0 M, 0 L
requirements_pip3.txt	12 minutes ago	11 minutes ago	0 C, 0 H, 0 M, 0 L

Rysunek 4: Zrzut ekranu z wyniku skanu repozytorium narzędziem Snyk[10].

7.2 Sugerowane kroki

Należałoby zintegrować projekt z narzędziem automatycznego formatowania kodu. Warto zamienić obecny sposób uruchamiania testów w Github Actions na inny, nie korzystający z Google Drive. Można do tego wykorzystać nowy podstawowy obraz aplikacji.

8 Baza danych JSON

Baza danych została stworzona, aby przechowywać informacje na temat poszczególnych przebiegów tj. informacje o wiązce, czas i ustawienia detektorów. Plik JSON powstał na podstawie pliku excel o rozszerzeniu .xlsx. Baza JSON została podzielona na 6 głównych węzłów - "metaData" (dane o przebiegu np. czas), "beam", "getNIMSettings", "conditions" (warunki pomiaru), "averageDaqRates" (pozyskiwanie danych np. częstość próbkowania). Konwersja plików xlsx została zautomatyzowana przez skrypt w języku python, który dba o poprawność danych (sprawdzenie typu) i pliku JSON. Skrypt posiada dwa pliki konfiguracyjne. Pierwszy z nich template.json określa typ danych ("num", "str") w danym polu, drugi - dict_json_x.json przechowuje nazwy kolumny z pliku .xlsx z której dane mają się znajdować w danym polu w pliku JSON. Cały skrypt wraz z opisem znajduje się w repozytorium github[11].

9 Podsumowanie

Projekt zespołowy realizowany został w semestrze zimowym roku akademickiego 2024/25 na Wydziale Fizyki Uniwersytetu Warszawskiego. Zrealizowano większość celów uzgodnionych z opiekunem projektu: udało się rozwinąć aplikacje do analizy danych z eksperymentu WarsawTPC zarówno poprzez połączenie istniejących już komponentów aplikacji jak i stworzenie nowych takich jak baza danych oraz metoda uruchamiania modeli Tensorflow w środowisku aplikacji. Nie wykonane zostało jednak zadanie rozwijania testów jednostkowych, ponadto opracowana metoda integracji z Tensorflow powinna zostać opakowana w nowy moduł.

Bibliografia

- [1] Wojciech Dominik. *Nuclear Astrophysics Studied With TPCs Operating in Gamma-Beams: Warsaw Active Target TPC*. URL: https://indico.duke.edu/event/1/contributions/33/attachments/28/39/nuclear_photonics_2023_WD.pdf. Wrz. 2023.
- [2] *Track reconstruction for TPC data with 2D projections readout*. URL: <https://github.com/akalinow/TPCReco> (term. wiz. 29.01.2025).
- [3] *TensorFlow C++ API Reference*. URL: https://www.tensorflow.org/api_docs/cc (term. wiz. 29.01.2025).
- [4] *GitHub Actions*. URL: <https://github.com/features/actions> (term. wiz. 29.01.2025).
- [5] *TPCReco fork*. URL: <https://github.com/pszyc/TPCReco> (term. wiz. 29.01.2025).
- [6] *Notion teamspace*. URL: <https://akalinow.notion.site/Teamspace-Home-3ffdcdf4b5c44a3687be98a8979a6249?pvs=4> (term. wiz. 29.01.2025).
- [7] *SOLID: The First 5 Principles of Object Oriented Design*. URL: <https://www.digitalocean.com/community/conceptual-articles/s-o-l-i-d-the-first-five-principles-of-object-oriented-design> (term. wiz. 29.01.2025).
- [8] *CppFlow library*. URL: <https://github.com/serizba/cppflow> (term. wiz. 29.01.2025).
- [9] *Find secrets with Gitleaks*. URL: <https://github.com/gitleaks/gitleaks> (term. wiz. 29.01.2025).
- [10] *Developer Security | Snyk*. URL: <https://snyk.io/> (term. wiz. 29.01.2025).
- [11] *Repozytorium ze skryptem tworzącym bazę JSON z pliku excel*. URL: <https://github.com/jkierzowski/zps/tree/main> (term. wiz. 04.02.2025).